MODUL PRAKTIKUM

INTEGRASI CHATBOT WHATSAPP

Praktikum Custom AI Chatbot dengan LLM dan RAG

Semester Ganjil 2026-2027



Mata Kuliah:

Kecerdasan Artifisial dan Penerapannya

Dosen Pengampu:

M. Nizar P. Ma'ady, S.Kom., M.IM., M.Kom.

Tim Asisten Praktikum:

Muhammad Baihaqi

Mohamad Fikri Isfahani

1. Pendahuluan

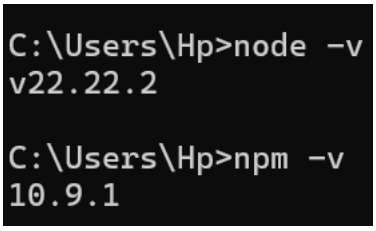
Modul ini terdiri atas dua kegiatan yang saling melengkapi. Kegiatan pertama adalah praktikum di kelas, yaitu merancang dan mengintegrasikan chatbot kecerdasan buatan (Artificial Intelligence) ke WhatsApp pada lingkungan lokal menggunakan Node.js, Express, whatsapp-web.js, arsitektur Retrieval-Augmented Generation (RAG), dan Groq AI. Kegiatan kedua adalah tugas lanjutan di rumah, yaitu membangun website katalog bisnis interaktif dan deploy ke Vercel.

Pemisahan kedua kegiatan ini didasarkan pada karakteristik arsitektur sistem. Chatbot WhatsApp membutuhkan proses latar belakang (background daemon/process) yang berjalan terus-menerus pada komputer lokal untuk menjaga sesi koneksi WhatsApp Web. Sementara itu, website katalog bisnis berbasis HTML, CSS, dan JavaScript statis dapat dideploy secara serverless ke cloud seperti Vercel sehingga dapat diakses online. Arsitektur utama chatbot yang dibangun pada praktikum ini yaitu:

- **1. WhatsApp Web Engine (whatsapp-web.js):** Menggunakan browser untuk mengontrol WhatsApp Web dan menghasilkan QR Code autentikasi.
- **2. Web Server & Dashboard (Express.js):** Menyediakan REST API untuk mengontrol status bot, menampilkan QR Code, dan menyajikan antarmuka visual pengguna.
- **3. Mesin Retrieval-Augmented Generation (lib/rag.js):** Membaca basis data produk lokal (.csv), melakukan klasifikasi intensi query, dan menyusun konteks katalog relevan.
- **4. Mesin Pemroses Bahasa Alami (Groq Cloud LLM):** Memanfaatkan Large Language Model (qwen/qwen3.8-27b atau llama-3.3-70b) melalui Groq SDK.

Requirement:

1. Download Node.js. Disarankan untuk menginstall versi LTS melalui situs resmi di https://nodejs.org/
2. Pastikan telah menyiapkan aplikasi WhatsApp yang siap untuk ditautkan dengan bot.
3. Untuk memastikan Node.js telah berhasil terinstall, cek CMD dengan mengetik node -v dan npm -v



2. Praktikum di Kelas: Membuat Chatbot WhatsApp Berbasis Web

Pada praktikum ini mahasiswa membangun project chatbot WhatsApp secara bertahap mulai dari inisialisasi lingkungan kerja, perancangan basis pengetahuan, implementasi mesin pencarian RAG, pembangunan backend server, hingga pembuatan dashboard web interaktif.

Langkah 1: Membuat Folder Project

Buka terminal Visual Studio Code, Command Prompt, atau Git Bash. Buat folder baru bernama chatbot-rag, kemudian masuk ke direktori tersebut.

```
● ● ●  Terminal
mkdir chatbot-rag
cd chatbot-rag
```

Folder chatbot-rag ini berfungsi sebagai wadah utama seluruh kode sumber, konfigurasi lingkungan, sesi autentikasi WhatsApp, dan berkas statis antarmuka dashboard.

Langkah 2: Inisialisasi Project Node.js

Jalankan perintah inisialisasi paket Node.js untuk membentuk berkas manifest package.json.

```
● ● ●  Terminal
npm init -y
```

Berkas package.json akan mencatat metadata project, dependensi modul yang digunakan, serta perintah script otomatisasi untuk menjalankan server backend.

Langkah 3: Memasang Library dan Dependensi

Pasang pustaka pihak ketiga yang dibutuhkan untuk komunikasi web, otomasi WhatsApp, integrasi kecerdasan buatan, dan environment variables.

```
● ● ●  Terminal
npm install express whatsapp-web.js qrcode-terminal groq-sdk dotenv cors body-parser
npm install --save-dev nodemon
```

Langkah 4: Membuat Struktur Folder dan File

Buka folder chatbot-rag di Visual Studio Code. Bentuklah struktur projek berikut:

```
● ● ●  Struktur Project
chatbot-rag/
├── server-v2.js         # Backend Express, logika bot WhatsApp, & integrasi LLM
├── .env                 # Kredensial API Groq, konfigurasi model, & batas token
├── nodemon.json         # Konfigurasi ignore file watcher agar nodemon stabil
├── knowledge.json       # Basis pengetahuan FAQ manual & informasi kontak toko
├── package.json         # Manifest dependensi & script runner
├── public/              # Berkas frontend antarmuka dashboard monitoring web
```

```
│   ├── index.html       # Struktur HTML dashboard, status bot, & panel kuota
│   ├── app.js           # Logika fetch API status, QR code, kuota, & aksi bot
│   └── style.css        # Tata letak CSS modern & badge indikator responsif
├── data/                # Basis data produk lokal
│   └── products.csv     # Katalog 8 produk perhiasan terstruktur 4 kategori
└── lib/                 # Modul pemrosesan cerdas lokal
    └── rag.js           # Mesin scoring relevansi & generator konteks katalog
```

Langkah 5: Konfigurasi File .env

Buat berkas .env pada direktori utama project. Berkas ini menyimpan kunci rahasia API Groq, penentuan model AI, batas token, dan port server. Dapatkan API key gratis melalui web resmi https://console.groq.com/keys untuk mendapatkan api key lalu masuk menggunakan akun yang masing-masing. Buat API Key baru (Create API Key) dan beri nama. Pilih masa aktif (expired) selama 90 days, lalu simpan kode API tersebut dengan baik.

```
● ● ●  .env
GROQ_API_KEY=gsk_isi_api_key_groq_kalian_disini
GROQ_MODEL=qwen/qwen3.8-27b
GROQ_MAX_TOKENS=1500
RAG_TOP_K=8
PORT=3001
AUTO_START_BOT=false
```

Langkah 6: Konfigurasi File nodemon.json (Pencegahan Crash Loop)

Saat WhatsApp Web Client berjalan, pustaka whatsapp-web.js secara berkala menulis token autentikasi, status sinkronisasi, dan berkas lock ke dalam folder .wwebjs_auth/ dan .wwebjs_cache/. Tanpa konfigurasi pengabaian (ignore), nodemon akan mendeteksi perubahan berkas tersebut dan me-restart server berulang kali (crash loop).

```
● ● ●  nodemon.json
{
  "watch": [
    "server-v2.js",
    "lib/",
    ".env"
  ],
  "ignore": [
    ".wwebjs_auth/**",
    ".wwebjs_cache/**",
    "public/**",
    "knowledge.json",
    "data/**",
    "node_modules/**"
  ],
  "ext": "js,json,env"
}
```

Langkah 7: Konfigurasi Scripts pada package.json

Buka package.json dan tambahkan script otomatisasi pada bagian 'scripts'.

```
● ● ●  package.json
```

```
{
  "name": "chatbot-rag",
  "version": "2.0.0",
  "scripts": {
    "start": "node server-v2.js",
    "dev": "nodemon server-v2.js"
  }
}
```

Langkah 8: Menyiapkan Basis Data Produk (data/products.csv)

Buat folder data dan isi berkas products.csv dengan katalog 8 produk perhiasan yang terbagi ke dalam 4 kategori (Gelang, Cincin, Kalung, dan Anting).

data/products.csv

```
nama,kategori,harga,deskripsi
Gelang Emas,Gelang,1500000,Gelang emas model sederhana
Cincin Emas,Cincin,2000000,Cincin emas untuk pria dan wanita
Kalung Emas,Kalung,2500000,Kalung emas dengan desain elegan
Anting Emas,Anting,1200000,Anting emas ringan untuk penggunaan harian
Gelang Berlian Charm,Gelang,3200000,Gelang rantai emas putih dengan liontin berlian mungil
Cincin Kawin Platinum,Cincin,4500000,Sepasang cincin kawin elegan bahan platinum anti karat
Kalung Mutiara Air Laut,Kalung,3800000,Kalung mutiara air laut asli kilau alami dengan
pengait emas
Anting Permata Safir,Anting,2800000,Anting jepit permata safir biru royal bingkai emas
kuning
```

Langkah 9: Menyiapkan Basis Pengetahuan FAQ & Kontak Toko (knowledge.json)

Buat berkas knowledge.json untuk menangani pertanyaan seputar jam operasional, alamat fisik, prosedur pemesanan, dan kontak resmi toko.

knowledge.json

```
{
  "keywords": {
    "jam buka": "jam buka",
    "alamat": "alamat",
    "lokasi": "alamat",
    "cara pesan": "cara pesan",
    "order": "cara pesan",
    "kontak": "kontak"
  },
  "responses": {
    "jam buka": "Toko kami buka setiap hari Senin - Sabtu pukul 09.00 - 20.00 WIB, dan
Minggu pukul 10.00 - 17.00 WIB.",
    "alamat": "Toko kami beralamat di Jl. Perhiasan No. 88, Pusat Emas & Permata, Jakarta.",
    "cara pesan": "Untuk pemesanan, silakan sebutkan nama produk yang diinginkan atau pilih
dari katalog kami, lalu konfirmasi pesanan melalui chat ini.",
    "kontak": "Informasi Kontak Toko Aura Jewelry & Gold Atelier:\n📍 *Alamat:* Jl.
Perhiasan No. 88, Pusat Emas & Permata, Jakarta\n📞 *Telepon / WA:* (021) 555-0199 / Chat
WhatsApp ini\n⏰ *Jam Operasional:* Senin - Sabtu (09.00 - 20.00 WIB), Minggu (10.00 -
17.00 WIB)"
  }
}
```

Langkah 10: Mengembangkan Retrieval-Augmented Generation (lib/rag.js)

RAG bertanggung jawab mengambil data relevan dari data/products.csv sebelum dikirim ke Groq LLM.

lib/rag.js

```
const fs = require("fs");
const path = require("path");

class RAGEngine {
  constructor(csvPath, options = {}) {
    this.csvPath = csvPath || path.join(__dirname, "../data/products.csv");
    this.topK = options.topK || Number(process.env.RAG_TOP_K || 8);
    this.products = [];
    this.loadData();
  }

  loadData() {
    try {
      if (!fs.existsSync(this.csvPath)) {
        console.warn(`[RAG] File CSV tidak ditemukan di: ${this.csvPath}`);
        this.products = [];
        return;
      }

      const content = fs.readFileSync(this.csvPath, "utf-8");
      const lines = content.trim().split(/\r?\n/);
      if (lines.length < 2) {
        this.products = [];
        return;
      }

      const headers = lines[0].split(",").map(h => h.trim().toLowerCase());
      this.products = lines.slice(1).map(line => {
        // Handle basic comma separation
        const cols = line.split(",").map(c => c.trim());
        const row = {};
        headers.forEach((h, idx) => {
          row[h] = cols[idx] || "";
        });
        return row;
      });

      console.log(`[RAG] Berhasil memuat ${this.products.length} produk dari CSV`);
    } catch (err) {
      console.error("[RAG] Gagal memuat data CSV:", err);
      this.products = [];
    }
  }

  formatCurrency(numStr) {
    const num = Number(numStr);
    if (isNaN(num)) return numStr;
    return new Intl.NumberFormat("id-ID", {
      style: "currency",
      currency: "IDR",
      maximumFractionDigits: 0
    }).format(num);
```

```
  }

  // Deteksi apakah query hanya berupa sapaan/salam tanpa menanyakan produk
  isGreetingQuery(query) {
    if (!query) return false;
    const q = query.toLowerCase().trim();
    const greetings = [
      "halo", "hai", "hi", "hey", "hello", "p", "ping", "test",
      "pagi", "selamat pagi", "siang", "selamat siang", "sore", "selamat sore",
      "malam", "selamat malam", "assalamualaikum", "assalamu'alaikum",
      "assalamu alaikum", "permisi"
    ];
    if (greetings.includes(q)) return true;

    // Deteksi salam dengan embel-embel (misal: "halo kak", "hai min") tanpa menyebut produk
    if
(/^(halo|hai|hi|hey|hello|selamat\s+(pagi|siang|sore|malam)|assalamu\s*alaikum)\b/i.test(q))
{
      const productKeywords = ["produk", "harga", "katalog", "jual", "beli", "gelang",
"cincin", "kalung", "anting", "emas", "berlian"];
      return !productKeywords.some(pk => q.includes(pk));
    }
    return false;
  }

  // Deteksi apakah query meminta seluruh katalog atau gambaran umum produk
  isAllProductsQuery(query) {
    if (!query) return false;
    const q = query.toLowerCase().trim();

    // Jika pengguna menanyakan kategori atau produk spesifik, BUKAN all products query
    const specificTerms = ["gelang", "cincin", "kalung", "anting", "warna", "ukuran",
"karat", "bahan", "harga", "promo", "rekomendasi"];
    if (specificTerms.some(term => q.includes(term))) {
      return false;
    }

    const allPatterns = [
      /^semua(\s+produk(nya)?)?$/,
      /^(apa|ada)\s+(aja|saja)(\s+produk(nya)?)?$/,
      /^produk\s+(apa|ada)\s+(aja|saja)$/,
      /^katalog(\s+produk)?$/,
      /^daftar\s+produk$/,
      /^list\s+produk$/,
      /^koleksi(\s+lengkap)?$/,
      /^semuanya$/,
      /^jelasin\s+produk(nya)?$/,
      /^daftar\s+harga$/
    ];
    return allPatterns.some(pattern => pattern.test(q)) || q === "semua produk" || q ===
"katalog lengkap";
  }

  // Cari produk relevan berdasarkan kata kunci atau kategori
  search(query, k = this.topK) {
    if (!query || this.products.length === 0) return [];

    const normalized = query.toLowerCase().trim();
```

```
    // 1. Jika query hanya berupa sapaan (halo, hai, dsb.), jangan tampilkan produk apapun
    if (this.isGreetingQuery(normalized)) {
      return [];
    }

    // 2. Jika pengguna meminta "semua produk", "apa aja produknya", "katalog", dsb.
    if (this.isAllProductsQuery(normalized)) {
      return this.products; // Kembalikan semua 8 produk dari CSV
    }

    // 3. Jika pengguna menyebutkan kategori spesifik (contoh: "kalung", "cincin", "gelang",
"anting")
    const categories = ["gelang", "cincin", "kalung", "anting"];
    const matchedCategory = categories.find(cat => normalized.includes(cat));

    // Ekstrak token pencarian
    const queryTokens = normalized
      .replace(/[^\w\s]/g, " ")
      .split(/\s+/)
      .filter(t => t.length > 2);

    // Filter produk berdasarkan kategori jika ada
    let candidateProducts = this.products;
    if (matchedCategory) {
      candidateProducts = this.products.filter(p => p.kategori && p.kategori.toLowerCase()
=== matchedCategory);
    }

    if (queryTokens.length === 0) {
      return candidateProducts;
    }

    // Skor produk kandidat
    const scored = candidateProducts.map(p => {
      let score = 0;
      const searchableText = `${p.nama} ${p.kategori} ${p.deskripsi}
${p.harga}`.toLowerCase();

      for (const token of queryTokens) {
        if (p.nama && p.nama.toLowerCase().includes(token)) score += 8;
        if (p.kategori && p.kategori.toLowerCase().includes(token)) score += 5;
        if (p.deskripsi && p.deskripsi.toLowerCase().includes(token)) score += 3;
        if (searchableText.includes(token)) score += 1;
      }

      return { product: p, score };
    })
    .filter(item => item.score > 0)
    .sort((a, b) => b.score - a.score);

    if (scored.length > 0) {
      return scored.slice(0, k).map(item => item.product);
    }

    // Jika ada kategori yang cocok walau token lain tidak cocok, kembalikan produk kategori
tersebut
```

```
    if (matchedCategory && candidateProducts.length > 0) {
      return candidateProducts;
    }

    // Jika tidak ada produk yang cocok sama sekali
    return [];
  }

  // Buat context teks untuk prompt Groq LLM dengan format terstruktur
  buildContext(query) {
    const relevantProducts = this.search(query);
    if (!relevantProducts || relevantProducts.length === 0) return "";

    // Kelompokkan produk berdasarkan kategori agar konteks rapi
    const grouped = {};
    for (const p of relevantProducts) {
      const cat = p.kategori || "Lainnya";
      if (!grouped[cat]) grouped[cat] = [];
      grouped[cat].push(p);
    }

    let output = "Daftar Katalog Produk Toko Resmi:\n";
    for (const [category, items] of Object.entries(grouped)) {
      output += `\n[Kategori: ${category}]\n`;
      items.forEach(p => {
        const hargaFmt = this.formatCurrency(p.harga);
        output += `• ${p.nama} - Harga: ${hargaFmt}\n  Deskripsi: ${p.deskripsi}\n`;
      });
    }

    return output.trim();
  }
}

module.exports = RAGEngine;
```

Langkah 11: Mengembangkan Backend Server Utama (server-v2.js)

File server-v2.js merupakan inti arsitektur dari keseluruhan sistem chatbot WhatsApp. Berkas ini mengintegrasikan server Express, modul RAG (lib/rag.js), basis pengetahuan (knowledge.json), client WhatsApp Web (whatsapp-web.js), integrasi Groq Cloud LLM SDK.

● ● ● server-v2.js (Source Code Lengkap)

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const bodyParser = require("body-parser");
const { Client, LocalAuth } = require("whatsapp-web.js");
const qrcode = require("qrcode-terminal");
const Groq = require("groq-sdk");
const fs = require("fs");
const path = require("path");
const RAGEngine = require("./lib/rag");

const app = express();
const PORT = process.env.PORT || 3001;
```

```
app.use(cors());
app.use(bodyParser.json());
app.use(express.static(path.join(__dirname, "public")));

// Inisialisasi Groq SDK
let groq = null;
if (process.env.GROQ_API_KEY && process.env.GROQ_API_KEY !== "isi_api_key_groq") {
  groq = new Groq({ apiKey: process.env.GROQ_API_KEY });
}

// Inisialisasi RAG Engine
const ragEngine = new RAGEngine(path.join(__dirname, "data/products.csv"));

// Load Knowledge Base
const KNOWLEDGE_PATH = path.join(__dirname, "knowledge.json");
let knowledge = { keywords: {}, responses: {} };
function loadKnowledge() {
  try {
    if (fs.existsSync(KNOWLEDGE_PATH)) {
      knowledge = JSON.parse(fs.readFileSync(KNOWLEDGE_PATH, "utf-8"));
    }
  } catch (err) {
    console.error("Gagal membaca knowledge.json:", err);
  }
}
loadKnowledge();

// State WhatsApp Bot
let client = null;
let qrCodeData = null;
let isReady = false;
let isInitializing = false;
let isCleaning = false;

// ================= KUOTA & RATE LIMIT GROQ AI =================
let groqQuota = {
  model: process.env.GROQ_MODEL || "qwen/qwen3.8-27b",
  remainingRequests: null,
  limitRequests: null,
  remainingTokens: null,
  limitTokens: null,
  resetRequests: null,
  lastChecked: null,
  configured: Boolean(process.env.GROQ_API_KEY && process.env.GROQ_API_KEY !==
"isi_api_key_groq")
};

function updateGroqQuotaFromHeaders(headers, modelName) {
  if (!headers) return;
  groqQuota.model = modelName || process.env.GROQ_MODEL || "qwen/qwen3.8-27b";
  const remReq = headers.get ? headers.get("x-ratelimit-remaining-requests") : headers["x-
ratelimit-remaining-requests"];
  const limReq = headers.get ? headers.get("x-ratelimit-limit-requests") : headers["x-
ratelimit-limit-requests"];
  const remTok = headers.get ? headers.get("x-ratelimit-remaining-tokens") : headers["x-
ratelimit-remaining-tokens"];
  const limTok = headers.get ? headers.get("x-ratelimit-limit-tokens") : headers["x-ratelimit-
limit-tokens"];
```

```
  const rstReq = headers.get ? headers.get("x-ratelimit-reset-requests") : headers["x-
ratelimit-reset-requests"];

  if (remReq) groqQuota.remainingRequests = Number(remReq);
  if (limReq) groqQuota.limitRequests = Number(limReq);
  if (remTok) groqQuota.remainingTokens = Number(remTok);
  if (limTok) groqQuota.limitTokens = Number(limTok);
  if (rstReq) groqQuota.resetRequests = rstReq;
  groqQuota.lastChecked = new Date().toLocaleTimeString("id-ID");
  groqQuota.configured = true;
}

async function fetchGroqQuotaFresh() {
  if (!groq) {
    if (process.env.GROQ_API_KEY && process.env.GROQ_API_KEY !== "isi_api_key_groq") {
      groq = new Groq({ apiKey: process.env.GROQ_API_KEY });
    } else {
      return groqQuota;
    }
  }

  const modelToUse = process.env.GROQ_MODEL || "qwen/qwen3.8-27b";
  try {
    const { response } = await groq.chat.completions.create({
      model: modelToUse,
      messages: [{ role: "user", content: "ping" }],
      max_tokens: 1
    }).withResponse();

    updateGroqQuotaFromHeaders(response.headers, modelToUse);
  } catch (err) {
    console.warn("Gagal mengecek kuota Groq:", err.message);
  }
  return groqQuota;
}

// ================= CONVERSATION HISTORY (MULTI-TURN MEMORY) =================
// Menyimpan riwayat percakapan per nomor pengirim (msg.from)
// Menggunakan sliding window (maks 6 pesan: 3 pertanyaan user & 3 respon bot)
// dan auto-expire setelah 20 menit tidak aktif.
const conversationHistory = new Map();
const MAX_HISTORY_MESSAGES = 6;
const SESSION_TTL_MS = 20 * 60 * 1000; // 20 menit

function getUserHistory(senderId) {
  if (!senderId) return [];
  const session = conversationHistory.get(senderId);
  const now = Date.now();

  if (!session) return [];

  // Jika sesi sudah tidak aktif lebih dari 20 menit, reset
  if (now - session.lastActive > SESSION_TTL_MS) {
    conversationHistory.delete(senderId);
    return [];
  }

  return session.messages || [];
```

```
}

function appendUserHistory(senderId, userMsg, botReply) {
  if (!senderId) return;
  const now = Date.now();
  let session = conversationHistory.get(senderId);

  if (!session || (now - session.lastActive > SESSION_TTL_MS)) {
    session = { messages: [], lastActive: now };
  }

  session.messages.push({ role: "user", content: userMsg });
  session.messages.push({ role: "assistant", content: botReply });

  // Batasi hanya menyimpan pesan terakhir
  if (session.messages.length > MAX_HISTORY_MESSAGES) {
    session.messages = session.messages.slice(-MAX_HISTORY_MESSAGES);
  }

  session.lastActive = now;
  conversationHistory.set(senderId, session);
}

function initializeClient() {
  if (client) return client;

  client = new Client({
    authStrategy: new LocalAuth({
      clientId: "chatbot-rag"
    }),
    puppeteer: {
      headless: true,
      args: [
        "--no-sandbox",
        "--disable-setuid-sandbox",
        "--disable-dev-shm-usage",
        "--disable-accelerated-2d-canvas",
        "--no-first-run",
        "--no-zygote",
        "--disable-gpu"
      ]
    }
  });

  client.on("qr", qr => {
    qrCodeData = qr;
    isInitializing = false;
    console.log("\n================ KODE QR WHATSAPP ================");
    qrcode.generate(qr, { small: true });
    console.log("Pindai QR code di atas atau melalui web dashboard.");
    console.log("==================================================\n");
  });

  client.on("authenticated", () => {
    console.log("Autentikasi WhatsApp berhasil.");
  });

  client.on("auth_failure", msg => {
```

```
    console.error("Autentikasi WhatsApp gagal:", msg);
    isInitializing = false;
  });

  client.on("ready", () => {
    isReady = true;
    isInitializing = false;
    qrCodeData = null;
    console.log("Bot WhatsApp siap digunakan");
  });

  client.on("disconnected", reason => {
    console.log("WhatsApp terputus:", reason);
    isReady = false;
    qrCodeData = null;
    client = null;
  });

  // Handler Pesan Pribadi
  client.on("message", async msg => {
    try {
      if (msg.fromMe || !msg.body) return;

      const isPersonalChat =
        msg.from.endsWith("@c.us") ||
        msg.from.endsWith("@lid");
      const isNotStatus = !msg.from.endsWith("@status");

      if (!isPersonalChat || !isNotStatus) {
        console.log("Pesan grup/status diabaikan:", msg.from);
        return;
      }

      const message = msg.body.trim();
      console.log(`[Pesan Masuk dari ${msg.from}]: ${message}`);

      const reply = await getAIResponse(msg.from, message);
      await msg.reply(reply);
      console.log(`[Balasan Terkirim]: ${reply}`);
    } catch (err) {
      console.error("Error memproses pesan:", err);
      try {
        await msg.reply("Mohon maaf, saat ini sistem asisten sedang mengalami gangguan teknis.
Silakan coba kembali sesaat lagi.");
      } catch (replyErr) {
        console.error("Gagal mengirim pesan error fallback:", replyErr);
      }
    }
  });

  return client;
}

// Helper Format Teks untuk WhatsApp (Bold *teks*, normalisasi spasi)
function formatWhatsAppMessage(text) {
  if (!text) return "";
  // 1. Ubah markdown bold double asterisks **teks** menjadi single asterisk *teks* untuk
WhatsApp
```

```
  let formatted = text.replace(/\*\*(.*?)\*\*/g, "*$1*");
  // 2. Pastikan tidak ada lebih dari 2 baris kosong berturut-turut
  formatted = formatted.replace(/\n{3,}/g, "\n\n").trim();
  return formatted;
}

// Helper deteksi sapaan/salam murni
function isGreetingMessage(text) {
  if (!text) return false;
  const q = text.toLowerCase().trim();
  const greetings = [
    "halo", "hai", "hi", "hey", "hello", "hei", "p", "ping", "test",
    "pagi", "selamat pagi", "siang", "selamat siang", "sore", "selamat sore",
    "malam", "selamat malam", "assalamualaikum", "assalamu'alaikum",
    "assalamu alaikum", "permisi", "sampurasun"
  ];
  if (greetings.includes(q)) return true;

  if
(/^(halo|hai|hi|hey|hello|selamat\s+(pagi|siang|sore|malam)|assalamu\s*alaikum|permisi)\b/i.te
st(q)) {
    const productKeywords = ["produk", "harga", "katalog", "jual", "beli", "gelang", "cincin",
"kalung", "anting", "emas", "berlian"];
    return !productKeywords.some(pk => q.includes(pk));
  }
  return false;
}

// Helper deteksi ucapan terima kasih
function isGratitudeMessage(text) {
  if (!text) return false;
  const q = text.toLowerCase().trim();
  const gratitudeList = [
    "terima kasih", "terimakasih", "makasih", "makasi", "thx", "thanks", "thank you",
    "matur nuwun", "nuhun", "syukron", "terima kasih banyak", "terimakasih banyak", "makasih
banyak"
  ];
  if (gratitudeList.includes(q)) return true;

  return
/^(terima\s*kasih|makasih|makasi|thanks|thank\s*you|thx|matur\s*nuwun|nuhun|syukron)\b/i.test(
q);
}

// Logika Respons AI & Basis Pengetahuan dengan Conversation History
async function getAIResponse(senderId, message) {
  const normalized = message.toLowerCase().trim();

  // 1. Template Sistem Cepat: Sapaan Awal (Langsung dijawab tanpa memanggil Groq AI)
  if (isGreetingMessage(normalized)) {
    console.log("[Template Sapaan]: Menggunakan template sistem cepat (tanpa panggil AI)");
    const greetingReply = "Halo! Selamat datang di *Aura Jewelry & Gold Atelier*. ✨\n\nAda
yang bisa kami bantu seputar koleksi perhiasan kami hari ini? 😊";
    if (senderId) appendUserHistory(senderId, message, greetingReply);
    return greetingReply;
  }

  // 2. Template Sistem Cepat: Ucapan Terima Kasih (Langsung dijawab tanpa memanggil Groq AI)
```

```
  if (isGratitudeMessage(normalized)) {
    console.log("[Template Terima Kasih]: Menggunakan template sistem cepat (tanpa panggil
AI)");
    const gratitudeReply = "Sama-sama! Senang bisa membantu Anda di *Aura Jewelry & Gold
Atelier*. ✨\n\nJika ada pertanyaan lain atau ingin melakukan pemesanan, silakan hubungi kami
kembali ya! 😊";
    if (senderId) appendUserHistory(senderId, message, gratitudeReply);
    return gratitudeReply;
  }

  // 3. Cek respons manual langsung dari knowledge.json
  if (knowledge.responses && knowledge.responses[normalized]) {
    console.log("[Knowledge Match]: Menggunakan jawaban manual keyword");
    const manualReply = formatWhatsAppMessage(knowledge.responses[normalized]);
    if (senderId) appendUserHistory(senderId, message, manualReply);
    return manualReply;
  }

  // 4. Cek apakah ada keyword alias di knowledge.keywords
  if (knowledge.keywords) {
    for (const [kw, targetKey] of Object.entries(knowledge.keywords)) {
      if (normalized.includes(kw.toLowerCase())) {
        if (knowledge.responses && knowledge.responses[targetKey]) {
          console.log(`[Knowledge Keyword Match]: Cocok dengan keyword '${kw}'`);
          const manualReply = formatWhatsAppMessage(knowledge.responses[targetKey]);
          if (senderId) appendUserHistory(senderId, message, manualReply);
          return manualReply;
        }
      }
    }
  }

  // 5. Ambil riwayat percakapan sebelumnya untuk nomor pengguna ini
  const history = senderId ? getUserHistory(senderId) : [];

  // 6. RAG Context dari CSV produk
  // Prioritaskan pencarian langsung dari pesan pengguna terkini agar riwayat bot sebelumnya
tidak mengontaminasi kata kunci
  let ragContext = ragEngine.buildContext(message);

  // Jika pesan pengguna sangat pendek (referensi dialog lanjutan seperti "berapa harganya?"),
gunakan konteks pertanyaan user sebelumnya
  if (!ragContext && history.length > 0) {
    const prevUserQueries = history
      .filter(h => h.role === "user")
      .slice(-2)
      .map(h => h.content)
      .join(" ");
    ragContext = ragEngine.buildContext(`${prevUserQueries} ${message}`);
  }

  // 7. Panggil Groq AI untuk Pertanyaan Produk / Informasi Lanjutan
  try {
    if (!groq) {
      // Inisialisasi ulang jika API key baru saja diset
      if (process.env.GROQ_API_KEY && process.env.GROQ_API_KEY !== "isi_api_key_groq") {
        groq = new Groq({ apiKey: process.env.GROQ_API_KEY });
      } else {
        return "Halo! Terima kasih telah menghubungi kami. Saat ini sistem sedang dalam mode
```

```
offline atau API Key Groq belum dikonfigurasi pada file .env.";
      }
    }

    const systemPrompt = `Anda adalah konsultan perhiasan resmi dari "Aura Jewelry & Gold Atelier", toko perhiasan dan emas terpercaya. Jawab dalam Bahasa Indonesia yang ramah, hangat, natural, to the point, dan sangat rapi.

PRINSIP KEAKURATAN DATA PRODUK (BERPIJAK KETAT PADA KATALOG RESMI):
1. Keaslian & Kebenaran Data:
   - Jawab HANYA berdasarkan informasi produk resmi yang tertera pada Konteks Katalog di bawah.
   - JANGAN PERNAH mengarang, berspekulasi, atau berasumsi tentang spesifikasi, varian warna, atau bahan yang tidak ada di katalog.
   - JANGAN PERNAH mencampuradukkan deskripsi antar produk (misal: "pengait emas" adalah milik Kalung Mutiara Air Laut, BUKAN milik Kalung Emas).
   - Karakteristik material perhiasan resmi toko:
     • *Gelang Emas*, *Cincin Emas*, *Kalung Emas*, *Anting Emas*: Material emas kuning klasik asli berkilau elegan.
     • *Gelang Berlian Charm*: Rantai emas putih dengan liontin berlian mungil.
     • *Cincin Kawin Platinum*: Bahan platinum anti karat yang elegan dan tahan lama.
     • *Kalung Mutiara Air Laut*: Mutiara air laut asli kilau alami dengan pengait emas.
     • *Anting Permata Safir*: Permata safir biru royal dengan bingkai emas kuning.
2. Jangan Pernah Memecah Karakter (No Fourth-Wall Breaking):
   - JANGAN PERNAH menyebut istilah internal/sistem seperti: "di file CSV", "deskripsi katalog tertulis secara umum", "di sistem kami...", atau "karena deskripsi tidak menyebutkan...".
   - Tampil selayaknya konsultan perhiasan profesional toko yang percaya diri dan memahami koleksi tokonya.
3. Jawab Ringkas, To-The-Point & Jangan Menguliahi:
   - Jawab pertanyaan spesifik (misal warna, harga, atau ukuran) secara langsung dalam 1-2 paragraf singkat.
   - JANGAN memberikan kuliah atau penjelasan teoritis panjang tentang emas.
   - Jika pelanggan menanyakan varian khusus, dimensi fisik (seperti diameter, berat), atau hal di luar katalog, jangan gunakan kata "hubungi tim kami", melainkan sertakan informasi kontak resmi toko secara langsung.

ATURAN PENGGANTIAN FRASA "TIM KAMI" / "CUSTOMER SERVICE" (SANGAT PENTING):
- DILARANG KERAS menggunakan kalimat generik seperti:
  ❌ "hubungi tim kami"
  ❌ "tanyakan ke tim kami"
  ❌ "konfirmasi ke tim toko"
  ❌ "silakan hubungi tim kami melalui WhatsApp"
  ❌ "hubungi admin / customer service"
- SETIAP KALI pelanggan menanyakan informasi spesifik yang tidak tersedia di katalog (seperti detail dimensi fisik, diameter permata, berat gramasi, kustomisasi ukuran/warna, atau stok terkini), SELALU GANTI dengan menampilkan rincian *Informasi Kontak Toko* secara lengkap:
  📍 *Alamat:* Jl. Perhiasan No. 88, Pusat Emas & Permata, Jakarta
  📞 *Telepon / WA:* (021) 555-0199 / Chat WhatsApp ini
  ⏰ *Jam Operasional:* Senin - Sabtu (09.00 - 20.00 WIB), Minggu (10.00 - 17.00 WIB)

ATURAN SALAM & KATA "SELAMAT DATANG":
1. Penggunaan Kata "Selamat Datang":
   - HANYA ucapkan sapaan pembuka "Selamat datang di Aura Jewelry & Gold Atelier" jika pesan pelanggan berupa salam/sapaan awal (misal: "halo", "hai", "selamat pagi") DAN belum ada percakapan sebelumnya.
   - Jika percakapan SUDAH BERLANGSUNG (ada riwayat obrolan di atas) ATAU pesan pelanggan langsung menanyakan produk/hal lain (misal: "apa saja produknya", "warna kalung emas apa aja"), JANGAN PERNAH mengulangi kata "Selamat datang di...". Langsung jawab inti pertanyaan secara natural dan ramah.
```

```
2. Jika Pesan Pengguna HANYA Berupa Sapaan (misal: "halo", "hai", "hi", "pagi", "ping"):
   - Balas dengan sapaan ramah, hangat, dan natural (contoh: "Halo! Selamat datang di *Aura Jewelry & Gold Atelier*. ✨ Ada yang bisa kami bantu seputar koleksi perhiasan kami hari ini?").
   - DILARANG KERAS menggunakan frasa kaku atau aneh seperti:
     ❌ "Saya asisten virtual resmi kami"
     ❌ "Saya adalah bot virtual"
     ❌ "Saya asisten AI Anda"
   - Berbicaralah selayaknya konsultan perhiasan toko manusiawi yang santun dan profesional.
   - JANGAN PERNAH menampilkan daftar produk atau katalog jika pengguna HANYA menyapa.

PANDUAN FORMAT TAMPILAN WHATSAPP:
1. Format Teks WhatsApp:
   - Gunakan SATU bintang untuk cetak tebal, contoh: *teks tebal* (JANGAN gunakan dua bintang **teks**).
   - Gunakan bullet point (•) untuk mendaftar produk.
   - Gunakan emoji yang elegan (✨, 💎, 💍, 📿, 📍, 📞, ⏰) secukupnya untuk memberi aksen visual.
2. JANGAN PERNAH menggabungkan daftar produk ke dalam satu paragraf panjang yang padat. Setiap produk WAJIB ditulis terpisah dalam baris baru.
3. Jika pelanggan meminta "semua produk" atau "katalog", kelompokkan berdasarkan kategori (*Koleksi Gelang*, *Koleksi Cincin*, *Koleksi Kalung*, *Koleksi Anting*) lengkap dengan harga dan deskripsinya.
4. Pastikan seluruh kalimat jawaban dan kalimat penutup diselesaikan secara tuntas dan tidak terpotong di tengah kalimat.
5. Jika pelanggan menanyakan produk di luar katalog, jawab sopan dan sertakan kontak toko resmi:
   📍 *Alamat:* Jl. Perhiasan No. 88, Pusat Emas & Permata, Jakarta
   📞 *Telepon / WA:* (021) 555-0199 / Chat WhatsApp ini
   ⏰ *Jam Operasional:* Senin - Sabtu (09.00 - 20.00 WIB), Minggu (10.00 - 17.00 WIB)

${ragContext ? ragContext : "Konteks Katalog: (Tidak ada katalog produk yang diminta pada pertanyaan ini. Jika pelanggan hanya menyapa, jawab sapaan saja dengan ramah)."}`;

    // Susun pesan: system prompt + riwayat obrolan sebelumnya + pesan user terkini
    const messagesPayload = [
      { role: "system", content: systemPrompt },
      ...history,
      { role: "user", content: message }
    ];

    const candidateModels = [
      process.env.GROQ_MODEL || "qwen/qwen3.8-27b",
      "openai/gpt-oss-20b",
      "allam-2-7b"
    ];

    let lastError = null;
    for (const modelName of candidateModels) {
      try {
        const { data: completion, response } = await groq.chat.completions.create({
          model: modelName,
          messages: messagesPayload,
          max_tokens: Number(process.env.GROQ_MAX_TOKENS || 1500)
        }).withResponse();

        if (response && response.headers) {
          updateGroqQuotaFromHeaders(response.headers, modelName);
```

```
        }

        if (completion.choices && completion.choices[0] && completion.choices[0].message) {
          const rawAiReply = completion.choices[0].message.content;
          const aiReply = formatWhatsAppMessage(rawAiReply);
          if (senderId) {
            appendUserHistory(senderId, message, aiReply);
          }
          return aiReply;
        }
      } catch (err) {
        lastError = err;
        console.warn(`[Groq Model ${modelName}] Gagal: ${err.message}. Mencoba model
alternatif...`);
      }
    }

    throw lastError || new Error("Semua model Groq gagal merespon.");
  } catch (error) {
    console.error("Error Groq API:", error.message || error);
    if (ragContext) {
      // Fallback jika Groq error tetapi ada data RAG
      return formatWhatsAppMessage(`Halo! ✨ Berikut informasi produk dari katalog
kami:\n\n${ragContext}\nAda yang bisa kami bantu seputar produk di atas? 😊`);
    }
    return formatWhatsAppMessage(`Halo! ✨ Terima kasih telah menghubungi *Aura Jewelry & Gold
Atelier*.\n\nKami menyediakan berbagai koleksi perhiasan emas dan permata seperti *Gelang*,
*Cincin*, *Kalung*, dan *Anting*.\n\nAda yang bisa kami bantu seputar informasi produk atau
pemesanan hari ini? 😊`);
  }
}

// Helper Masking Nomor Telepon (Format: 08xxxxxx4digit)
function maskPhoneNumber(rawNumber) {
  if (!rawNumber) return null;
  let formatted = String(rawNumber).replace(/^62/, "08");
  if (formatted.length >= 8) {
    const prefix = formatted.slice(0, 2); // "08"
    const suffix = formatted.slice(-4);   // 4 digit terakhir
    const maskedLength = formatted.length - 6;
    return `${prefix}${"x".repeat(maskedLength)}${suffix}`;
  }
  return formatted;
}

// ================= API ENDPOINTS =================

// Status Bot
app.get("/api/bot/status", (req, res) => {
  let botNumber = null;
  if (isReady && client && client.info && client.info.wid) {
    botNumber = maskPhoneNumber(client.info.wid.user);
  }

  res.json({
    isReady,
    isInitializing,
    isCleaning,
```

```
    hasQRCode: Boolean(qrCodeData),
    botNumber
  });
});

// Kuota & Rate Limit Groq AI
app.get("/api/groq/quota", async (req, res) => {
  try {
    const isFresh = req.query.fresh === "true" || groqQuota.remainingRequests === null;
    if (isFresh) {
      await fetchGroqQuotaFresh();
    }
    res.json(groqQuota);
  } catch (err) {
    console.error("Error cek kuota Groq:", err);
    res.status(500).json({ error: err.message, ...groqQuota });
  }
});

// QR Code Data
app.get("/api/bot/qr", (req, res) => {
  res.json({ qr: qrCodeData });
});

// Start Bot
app.post("/api/bot/start", async (req, res) => {
  try {
    if (isReady) {
      return res.json({ success: true, message: "Bot sudah berjalan." });
    }
    if (isInitializing) {
      return res.json({ success: true, message: "Bot sedang proses inisialisasi..." });
    }

    isInitializing = true;
    const waClient = initializeClient();
    await waClient.initialize();
    res.json({ success: true });
  } catch (err) {
    isInitializing = false;
    console.error("Gagal memulai bot:", err);
    res.status(500).json({ success: false, error: err.message });
  }
});

// Stop Bot
app.post("/api/bot/stop", async (req, res) => {
  try {
    isCleaning = true;
    isReady = false;
    qrCodeData = null;
    if (client) {
      await client.destroy();
    }
    client = null;
    isCleaning = false;
    res.json({ success: true });
  } catch (err) {
    isCleaning = false;
```

```
    console.error("Gagal menghentikan bot:", err);
    res.status(500).json({ success: false, error: err.message });
  }
});

// Reset Sesi WhatsApp (Hapus sesi yang nyangkut / logout)
app.post("/api/bot/reset", async (req, res) => {
  try {
    isCleaning = true;
    isReady = false;
    isInitializing = false;
    qrCodeData = null;

    if (client) {
      try {
        await client.destroy();
      } catch (e) {
        console.warn("Peringatan saat destroy client:", e.message);
      }
      client = null;
    }

    const authDir = path.join(__dirname, ".wwebjs_auth");
    const cacheDir = path.join(__dirname, ".wwebjs_cache");

    if (fs.existsSync(authDir)) {
      fs.rmSync(authDir, { recursive: true, force: true });
      console.log("[Reset Session]: Folder .wwebjs_auth berhasil dihapus");
    }
    if (fs.existsSync(cacheDir)) {
      fs.rmSync(cacheDir, { recursive: true, force: true });
      console.log("[Reset Session]: Folder .wwebjs_cache berhasil dihapus");
    }

    conversationHistory.clear();
    console.log("[Reset Session]: Seluruh riwayat obrolan berhasil dibersihkan");

    isCleaning = false;
    res.json({ success: true, message: "Sesi WhatsApp berhasil dibersihkan. Anda dapat
memindai QR baru." });
  } catch (err) {
    isCleaning = false;
    console.error("Gagal mereset sesi bot:", err);
    res.status(500).json({ success: false, error: err.message });
  }
});

// Ambil data Basis Pengetahuan
app.get("/api/knowledge", (req, res) => {
  loadKnowledge();
  res.json(knowledge);
});

// Simpan Kata Kunci & Respons Manual ke Basis Pengetahuan
app.post("/api/knowledge", (req, res) => {
  try {
    const { keyword, response } = req.body;
    if (!keyword || !response) {
```

```
      return res.status(400).json({ success: false, error: "Keyword dan response wajib diisi."
});
    }

    const key = keyword.toLowerCase().trim();
    loadKnowledge();

    if (!knowledge.keywords) knowledge.keywords = {};
    if (!knowledge.responses) knowledge.responses = {};

    knowledge.keywords[key] = key;
    knowledge.responses[key] = response.trim();

    fs.writeFileSync(KNOWLEDGE_PATH, JSON.stringify(knowledge, null, 2), "utf-8");
    console.log(`[Knowledge Updated]: Keyword '${key}' berhasil disimpan.`);

    res.json({ success: true, data: { keyword: key, response: response.trim() } });
  } catch (err) {
    console.error("Gagal menyimpan basis pengetahuan:", err);
    res.status(500).json({ success: false, error: err.message });
  }
});

// Ambil daftar produk CSV
app.get("/api/products", (req, res) => {
  ragEngine.loadData();
  res.json({ products: ragEngine.products });
});

// Jalankan Server
const server = app.listen(PORT, () => {
  console.log(`Server berjalan di http://localhost:${PORT}`);
  if (process.env.AUTO_START_BOT === "true") {
    console.log("AUTO_START_BOT aktif, memulai inisialisasi bot WhatsApp...");
    isInitializing = true;
    const waClient = initializeClient();
    waClient.initialize().catch(err => {
      isInitializing = false;
      console.error("Auto start gagal:", err);
    });
  }
});

// Penutupan bersih saat server dihentikan (mencegah Chrome nyangkut)
async function gracefulShutdown(signal) {
  console.log(`\nMenerima sinyal ${signal}, membersihkan proses...`);
  if (client) {
    try {
      await client.destroy();
      console.log("Koneksi WhatsApp & browser Chrome berhasil ditutup.");
    } catch (e) {
      // Abaikan jika sudah tertutup
    }
  }
  server.close(() => {
    process.exit(0);
  });
  setTimeout(() => process.exit(0), 3000);
```

```
}

process.on("SIGINT", () => gracefulShutdown("SIGINT"));
process.on("SIGTERM", () => gracefulShutdown("SIGTERM"));
```

Langkah 12: Membuat Tampilan Dashboard Web (public/index.html)

Dashboard web berfungsi sebagai antarmuka pemantauan status koneksi bot WhatsApp, tampilan nomor WhatsApp terhubung yang disamarkan, kuota tracking API Groq, tombol reset sesi, penampil data katalog CSV, serta scanner kode QR:

• • • public/index.html

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bot WhatsApp Sederhana</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="container">
    <section class="header">
      <div class="header-title">
        <h1>Bot WhatsApp Sederhana</h1>
        <p class="subtitle">Integrasi Chatbot WhatsApp dengan Groq AI & RAG</p>
      </div>
      <div class="status-wrapper">
        <span class="status-indicator" id="statusDot"></span>
        <span id="statusText">Bot Offline</span>
      </div>
      <div class="btn-group">
        <button id="startBtn">Mulai Bot</button>
        <button id="stopBtn" class="hidden">Hentikan Bot</button>
        <button id="resetBtn" title="Hapus sesi WA yang nyangkut / logout">Reset Sesi
WA</button>
      </div>
    </section>

    <section id="qrSection" class="panel hidden">
      <h2>Pindai Kode QR</h2>
      <p>Buka WhatsApp pada ponsel Anda, buka menu <strong>Perangkat Tertaut</strong>, lalu
arahkan kamera ke kode QR berikut:</p>
      <div id="qrcode">
        <div class="loading-spinner">Menunggu kode QR dari WhatsApp...</div>
      </div>
    </section>

    <section id="readySection" class="panel hidden">
      <div class="ready-badge">
        <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2">
          <path d="M22 11.08V12a10 10 0 1 1-5.93-9.14"></path>
          <polyline points="22 4 12 14.01 9 11.01"></polyline>
        </svg>
        <div class="ready-content">
```

```
          <div class="ready-header-row">
            <h2>Bot Terhubung</h2>
            <span id="botNumberBadge" class="bot-number-badge">Nomor WA: <strong
id="botNumberText">-</strong></span>
          </div>
          <p>Bot siap menerima dan membalas pesan WhatsApp secara otomatis menggunakan Groq
AI & RAG.</p>
        </div>
      </div>
    </section>

    <!-- Panel Monitoring Kuota Groq AI -->
    <section class="panel" id="groqQuotaSection">
      <div class="quota-header">
        <div>
          <h2>Monitoring Kuota Groq AI</h2>
          <p class="section-desc">Informasi sisa kuota request, batas token (TPM), dan model
Groq AI yang sedang aktif.</p>
        </div>
        <button id="refreshQuotaBtn" class="btn-refresh" title="Perbarui kuota langsung dari
Groq">🔄 Cek Kuota</button>
      </div>

      <div class="quota-grid">
        <div class="quota-card">
          <div class="quota-card-label">Sisa Request</div>
          <div class="quota-card-value" id="groqReqRemaining">Memuat...</div>
          <div class="progress-bar-bg">
            <div class="progress-bar-fill" id="groqReqBar" style="width: 100%;"></div>
          </div>
          <div class="quota-card-sub" id="groqReqReset">Reset dalam: -</div>
        </div>

        <div class="quota-card">
          <div class="quota-card-label">Sisa Token (TPM)</div>
          <div class="quota-card-value" id="groqTokenRemaining">Memuat...</div>
          <div class="progress-bar-bg">
            <div class="progress-bar-fill tokens" id="groqTokenBar" style="width:
100%;"></div>
          </div>
          <div class="quota-card-sub">Batas Token Menit Ini</div>
        </div>

        <div class="quota-card">
          <div class="quota-card-label">Model Groq Digunakan</div>
          <div class="quota-card-value model-name" id="groqModelName">Memuat...</div>
          <div class="quota-card-sub" id="groqLastChecked">Pembaruan: Baru saja</div>
        </div>
      </div>
    </section>

    <section class="panel">
      <h2>Basis Pengetahuan (Manual Responses)</h2>
      <p class="section-desc">Tambahkan kata kunci dan respons kustom untuk pertanyaan umum
pelanggan.</p>
      <input id="keyword" placeholder="Kata kunci (contoh: jam buka, promo, lokasi)">
      <textarea id="response" placeholder="Respons manual yang akan dijawab oleh
bot"></textarea>
      <button id="saveBtn">Simpan Kata Kunci</button>
```

```
      <div id="saveNotice" class="notice hidden"></div>

      <div class="knowledge-preview">
        <h3>Daftar Kata Kunci Aktif:</h3>
        <div id="knowledgeList" class="knowledge-tags">
          <em>Memuat daftar kata kunci...</em>
        </div>
      </div>
    </section>

    <section class="panel">
      <h2>Katalog Produk (RAG Source)</h2>
      <p class="section-desc">Data produk dari <code>data/products.csv</code> yang dijadikan
konteks referensi oleh Groq AI.</p>
      <div class="table-responsive">
        <table id="productsTable">
          <thead>
            <tr>
              <th>Nama Produk</th>
              <th>Kategori</th>
              <th>Harga</th>
              <th>Deskripsi</th>
            </tr>
          </thead>
          <tbody id="productsBody">
            <tr><td colspan="4" style="text-align:center;">Memuat data produk...</td></tr>
          </tbody>
        </table>
      </div>
    </section>
  </main>

  <script src="app.js"></script>
</body>
</html>
```

Langkah 13: Menambahkan Styling Modern (public/style.css)

public/style.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial,
sans-serif;
  background: #eef2f5;
  color: #2c3e50;
  line-height: 1.5;
}

.container {
  max-width: 900px;
  margin: 40px auto;
  padding: 20px;
}

.header, .panel {
```

```
  background: white;
  padding: 28px;
  margin-bottom: 20px;
  border-radius: 12px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

.header {
  display: flex;
  flex-direction: column;
  gap: 16px;
}

.header-title h1 {
  margin: 0 0 6px 0;
  font-size: 26px;
  color: #1a252f;
}

.subtitle {
  margin: 0;
  color: #7f8c8d;
  font-size: 14px;
}

.status-wrapper {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 6px 14px;
  border-radius: 20px;
  background: #f8f9fa;
  width: fit-content;
  font-weight: 600;
  font-size: 14px;
}

.status-indicator {
  width: 10px;
  height: 10px;
  border-radius: 50%;
  background: #95a5a6;
  display: inline-block;
}

.status-indicator.ready {
  background: #27ae60;
  box-shadow: 0 0 8px #27ae60;
}

.status-indicator.waiting {
  background: #f39c12;
  box-shadow: 0 0 8px #f39c12;
}

.btn-group {
  display: flex;
  gap: 12px;
```

```
}

button {
  padding: 12px 24px;
  border: 0;
  border-radius: 8px;
  color: white;
  background: #2c3e50;
  font-weight: 600;
  font-size: 14px;
  cursor: pointer;
  transition: all 0.2s ease;
}

button:hover {
  background: #1a252f;
}

#startBtn {
  background: #27ae60;
}

#startBtn:hover {
  background: #219653;
}

#stopBtn {
  background: #e74c3c;
}

#stopBtn:hover {
  background: #c0392b;
}

#resetBtn {
  background: #7f8c8d;
}

#resetBtn:hover {
  background: #5d6d7e;
}

#saveBtn {
  background: #2980b9;
}

#saveBtn:hover {
  background: #2471a3;
}

.section-desc {
  color: #666;
  font-size: 14px;
  margin-top: -6px;
  margin-bottom: 16px;
}
```

```
input, textarea {
  display: block;
  width: 100%;
  margin: 12px 0;
  padding: 12px 14px;
  border: 1px solid #dcdfe6;
  border-radius: 8px;
  font-size: 14px;
  font-family: inherit;
  transition: border-color 0.2s;
}

input:focus, textarea:focus {
  outline: none;
  border-color: #3498db;
}

textarea {
  min-height: 120px;
  resize: vertical;
}

.hidden {
  display: none !important;
}

#qrcode {
  text-align: center;
  margin-top: 20px;
  padding: 20px;
  background: #fafafa;
  border-radius: 8px;
  min-height: 280px;
  display: flex;
  align-items: center;
  justify-content: center;
}

#qrcode img {
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

.loading-spinner {
  color: #7f8c8d;
  font-style: italic;
}

.ready-badge {
  display: flex;
  align-items: center;
  gap: 16px;
  background: #e8f8f0;
  padding: 16px;
  border-radius: 8px;
  border-left: 4px solid #27ae60;
  color: #1e7e34;
}
```

```
.ready-content {
  flex-grow: 1;
}

.ready-header-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 8px;
  margin-bottom: 4px;
}

.ready-badge h2 {
  margin: 0;
  font-size: 18px;
}

.bot-number-badge {
  background: #27ae60;
  color: white;
  padding: 3px 10px;
  border-radius: 14px;
  font-size: 13px;
  font-family: monospace, sans-serif;
  letter-spacing: 0.5px;
  box-shadow: 0 1px 4px rgba(39, 174, 96, 0.3);
}

/* Groq Quota Monitor Panel */
.quota-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  flex-wrap: wrap;
  gap: 12px;
  margin-bottom: 16px;
}

.btn-refresh {
  background: #f1f3f5;
  color: #2c3e50;
  border: 1px solid #dcdfe6;
  padding: 8px 14px;
  font-size: 13px;
}

.btn-refresh:hover {
  background: #e2e8f0;
  color: #1a252f;
}

.quota-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 16px;
  margin-top: 10px;
```

```
}

.quota-card {
  background: #f8fafc;
  border: 1px solid #e2e8f0;
  border-radius: 10px;
  padding: 16px;
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.quota-card-label {
  font-size: 12px;
  font-weight: 600;
  color: #64748b;
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.quota-card-value {
  font-size: 20px;
  font-weight: 700;
  color: #1e293b;
}

.quota-card-value.model-name {
  font-size: 15px;
  color: #3b82f6;
  font-family: monospace;
  word-break: break-all;
}

.progress-bar-bg {
  width: 100%;
  height: 6px;
  background: #e2e8f0;
  border-radius: 3px;
  overflow: hidden;
  margin: 4px 0;
}

.progress-bar-fill {
  height: 100%;
  background: #10b981;
  border-radius: 3px;
  transition: width 0.4s ease;
}

.progress-bar-fill.tokens {
  background: #6366f1;
}

.progress-bar-fill.warning {
  background: #f59e0b;
}

.progress-bar-fill.danger {
```

```
  background: #ef4444;
}

.quota-card-sub {
  font-size: 12px;
  color: #94a3b8;
}

.ready-badge p {
  margin: 0;
  font-size: 14px;
  color: #2d5a3c;
}

.notice {
  margin-top: 10px;
  padding: 10px;
  border-radius: 6px;
  font-size: 13px;
}

.notice.success {
  background: #d4edda;
  color: #155724;
}

.notice.error {
  background: #f8d7da;
  color: #721c24;
}

.knowledge-preview {
  margin-top: 24px;
  padding-top: 16px;
  border-top: 1px solid #edf2f7;
}

.knowledge-preview h3 {
  font-size: 15px;
  margin-bottom: 10px;
  color: #4a5568;
}

.knowledge-tags {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

.tag-item {
  background: #e2e8f0;
  color: #334155;
  padding: 6px 12px;
  border-radius: 16px;
  font-size: 12px;
  display: inline-flex;
  align-items: center;
  gap: 6px;
```

```
}

.table-responsive {
  overflow-x: auto;
  margin-top: 12px;
}

table {
  width: 100%;
  border-collapse: collapse;
  font-size: 13px;
}

table th, table td {
  padding: 10px 14px;
  border: 1px solid #e2e8f0;
  text-align: left;
}

table th {
  background: #f8fafc;
  font-weight: 600;
  color: #475569;
}

table tr:nth-child(even) {
  background: #fdfdfd;
}

@media (max-width: 600px) {
  .container {
    margin: 10px auto;
    padding: 10px;
  }
  .header, .panel {
    padding: 18px;
  }
}
```

Langkah 14: Mengisi Logika Interaktivitas Web (public/app.js)

Berfungsi untuk membaca status bot, merender kode QR secara langsung, memperbarui meter kuota Groq, menampilkan daftar produk RAG, dan menangani perintah reset sesi.

public/app.js

```
const API_URL = "/api";
const startBtn = document.getElementById("startBtn");
const stopBtn = document.getElementById("stopBtn");
const resetBtn = document.getElementById("resetBtn");
const statusText = document.getElementById("statusText");
const statusDot = document.getElementById("statusDot");
const qrSection = document.getElementById("qrSection");
const readySection = document.getElementById("readySection");
const botNumberText = document.getElementById("botNumberText");

const keywordInput = document.getElementById("keyword");
const responseInput = document.getElementById("response");
const saveBtn = document.getElementById("saveBtn");
```

```
const saveNotice = document.getElementById("saveNotice");
const knowledgeList = document.getElementById("knowledgeList");
const productsBody = document.getElementById("productsBody");

// Elemen Monitoring Kuota Groq
const refreshQuotaBtn = document.getElementById("refreshQuotaBtn");
const groqReqRemaining = document.getElementById("groqReqRemaining");
const groqReqBar = document.getElementById("groqReqBar");
const groqReqReset = document.getElementById("groqReqReset");
const groqTokenRemaining = document.getElementById("groqTokenRemaining");
const groqTokenBar = document.getElementById("groqTokenBar");
const groqModelName = document.getElementById("groqModelName");
const groqLastChecked = document.getElementById("groqLastChecked");

let currentQRCode = null;

// Cek status bot WhatsApp
async function checkBotStatus() {
  try {
    const response = await fetch(`${API_URL}/bot/status`);
    if (!response.ok) throw new Error("Gagal memeriksa status bot");
    const data = await response.json();

    if (data.isReady) {
      if (data.botNumber) {
        statusText.textContent = `Bot Connected (${data.botNumber})`;
        if (botNumberText) {
          botNumberText.textContent = data.botNumber;
        }
      } else {
        statusText.textContent = "Bot Connected";
        if (botNumberText) {
          botNumberText.textContent = "Terhubung";
        }
      }
      if (statusDot) {
        statusDot.className = "status-indicator ready";
      }
      startBtn.classList.add("hidden");
      stopBtn.classList.remove("hidden");
      readySection.classList.remove("hidden");
      qrSection.classList.add("hidden");
      currentQRCode = null;
    } else if (data.hasQRCode) {
      if (botNumberText) {
        botNumberText.textContent = "-";
      }
      statusText.textContent = "Waiting for QR Scan";
      if (statusDot) {
        statusDot.className = "status-indicator waiting";
      }
      qrSection.classList.remove("hidden");
      readySection.classList.add("hidden");
      startBtn.classList.add("hidden");
      stopBtn.classList.remove("hidden");
      loadQRCode();
    } else if (data.isInitializing) {
      statusText.textContent = "Menginisialisasi Bot WhatsApp...";
      if (statusDot) {
        statusDot.className = "status-indicator waiting";
```

```
      }
      startBtn.classList.add("hidden");
      stopBtn.classList.remove("hidden");
      qrSection.classList.remove("hidden");
      readySection.classList.add("hidden");
    } else {
      statusText.textContent = "Bot Offline";
      if (statusDot) {
        statusDot.className = "status-indicator";
      }
      startBtn.classList.remove("hidden");
      stopBtn.classList.add("hidden");
      readySection.classList.add("hidden");
      qrSection.classList.add("hidden");
      currentQRCode = null;
    }
  } catch (error) {
    statusText.textContent = "Server Offline (Pastikan backend berjalan)";
    if (statusDot) {
      statusDot.className = "status-indicator";
    }
    startBtn.classList.remove("hidden");
    stopBtn.classList.add("hidden");
  }
}

// Muat dan tampilkan gambar QR Code
async function loadQRCode() {
  try {
    const response = await fetch(`${API_URL}/bot/qr`);
    const data = await response.json();

    if (data.qr && data.qr !== currentQRCode) {
      currentQRCode = data.qr;
      const qrContainer = document.getElementById("qrcode");
      qrContainer.innerHTML = `<img width="280" height="280" alt="QR Code WhatsApp"
src="https://api.qrserver.com/v1/create-qr-
code/?size=280x280&data=${encodeURIComponent(data.qr)}">`;
    }
  } catch (error) {
    console.error("Gagal memuat QR Code:", error);
  }
}

// Handler Tombol Mulai Bot
startBtn.onclick = async () => {
  try {
    startBtn.disabled = true;
    startBtn.textContent = "Memulai...";
    await fetch(`${API_URL}/bot/start`, { method: "POST" });
  } catch (error) {
    console.error("Error memulai bot:", error);
    alert("Gagal mengirim perintah start ke server.");
  } finally {
    startBtn.disabled = false;
    startBtn.textContent = "Mulai Bot";
    checkBotStatus();
  }
};
```

```
// Handler Tombol Hentikan Bot
stopBtn.onclick = async () => {
  try {
    stopBtn.disabled = true;
    stopBtn.textContent = "Menghentikan...";
    await fetch(`${API_URL}/bot/stop`, { method: "POST" });
  } catch (error) {
    console.error("Error menghentikan bot:", error);
    alert("Gagal mengirim perintah stop ke server.");
  } finally {
    stopBtn.disabled = false;
    stopBtn.textContent = "Hentikan Bot";
    checkBotStatus();
  }
};

// Handler Tombol Reset Sesi Bot
if (resetBtn) {
  resetBtn.onclick = async () => {
    if (!confirm("Apakah Anda yakin ingin menghapus sesi WhatsApp? Anda harus scan QR
ulang.")) {
      return;
    }
    try {
      resetBtn.disabled = true;
      resetBtn.textContent = "Mereset...";
      const res = await fetch(`${API_URL}/bot/reset`, { method: "POST" });
      const data = await res.json();
      if (data.success) {
        alert(data.message || "Sesi berhasil dibersihkan!");
      } else {
        alert("Gagal mereset: " + (data.error || "Kesalahan server"));
      }
    } catch (error) {
      console.error("Error mereset bot:", error);
      alert("Gagal menghubungi endpoint reset.");
    } finally {
      resetBtn.disabled = false;
      resetBtn.textContent = "Reset Sesi WA";
      checkBotStatus();
    }
  };
}

// Handler Simpan Basis Pengetahuan
if (saveBtn) {
  saveBtn.onclick = async () => {
    const keyword = keywordInput.value.trim();
    const response = responseInput.value.trim();

    if (!keyword || !response) {
      showNotice("Kata kunci dan respons tidak boleh kosong!", "error");
      return;
    }

    try {
      saveBtn.disabled = true;
      saveBtn.textContent = "Menyimpan...";
      const res = await fetch(`${API_URL}/knowledge`, {
        method: "POST",
```

```
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ keyword, response })
      });
      const data = await res.json();

      if (data.success) {
        showNotice(`Kata kunci "${keyword}" berhasil disimpan!`, "success");
        keywordInput.value = "";
        responseInput.value = "";
        loadKnowledgeList();
      } else {
        showNotice(`Gagal menyimpan: ${data.error || "Kesalahan server"}`, "error");
      }
    } catch (err) {
      showNotice("Gagal menghubungi server.", "error");
    } finally {
      saveBtn.disabled = false;
      saveBtn.textContent = "Simpan Kata Kunci";
    }
  };
}

function showNotice(msg, type) {
  if (!saveNotice) return;
  saveNotice.textContent = msg;
  saveNotice.className = `notice ${type}`;
  setTimeout(() => {
    saveNotice.className = "notice hidden";
  }, 4000);
}

// Muat daftar kata kunci yang tersimpan
async function loadKnowledgeList() {
  if (!knowledgeList) return;
  try {
    const res = await fetch(`${API_URL}/knowledge`);
    const data = await res.json();
    const responses = data.responses || {};
    const keys = Object.keys(responses);

    if (keys.length === 0) {
      knowledgeList.innerHTML = "<em>Belum ada kata kunci kustom.</em>";
      return;
    }

    knowledgeList.innerHTML = keys
      .map(k => `<span class="tag-item"
title="${responses[k]}"><strong>${k}</strong></span>`)
      .join("");
  } catch (err) {
    knowledgeList.innerHTML = "<em>Gagal memuat basis pengetahuan.</em>";
  }
}

// Muat daftar produk dari CSV
async function loadProductsList() {
  if (!productsBody) return;
  try {
    const res = await fetch(`${API_URL}/products`);
```

```
    const data = await res.json();
    const products = data.products || [];

    if (products.length === 0) {
      productsBody.innerHTML = '<tr><td colspan="4" style="text-align:center;">Tidak ada
data produk.</td></tr>';
      return;
    }

    productsBody.innerHTML = products.map(p => {
      const harga = Number(p.harga);
      const formatted = isNaN(harga)
        ? p.harga
        : new Intl.NumberFormat("id-ID", { style: "currency", currency: "IDR",
maximumFractionDigits: 0 }).format(harga);

      return `
        <tr>
          <td><strong>${p.nama}</strong></td>
          <td><span class="tag-item">${p.kategori}</span></td>
          <td>${formatted}</td>
          <td>${p.deskripsi}</td>
        </tr>
      `;
    }).join("");
  } catch (err) {
    productsBody.innerHTML = '<tr><td colspan="4" style="text-align:center;color:red;">Gagal
memuat produk dari server.</td></tr>';
  }
}

// Muat data kuota & batas limit Groq AI
async function loadGroqQuota(isFresh = false) {
  try {
    const url = isFresh ? `${API_URL}/groq/quota?fresh=true` : `${API_URL}/groq/quota`;
    const res = await fetch(url);
    if (!res.ok) throw new Error("Gagal mengambil data kuota");
    const data = await res.json();

    if (data.configured) {
      if (groqModelName) groqModelName.textContent = data.model || "-";
      if (groqLastChecked) groqLastChecked.textContent = `Pembaruan: ${data.lastChecked ||
"Baru saja"}`;

      if (data.remainingRequests !== null && data.limitRequests !== null) {
        if (groqReqRemaining) {
          groqReqRemaining.textContent = `${Number(data.remainingRequests).toLocaleString()}
/ ${Number(data.limitRequests).toLocaleString()}`;
        }
        if (groqReqBar) {
          const pct = Math.max(0, Math.min(100, (data.remainingRequests /
data.limitRequests) * 100));
          groqReqBar.style.width = `${pct}%`;
          groqReqBar.className = pct < 20 ? "progress-bar-fill danger" : pct < 50 ?
"progress-bar-fill warning" : "progress-bar-fill";
        }
        if (groqReqReset && data.resetRequests) {
          groqReqReset.textContent = `Reset dalam: ${data.resetRequests}`;
        }
```

```
      } else {
        if (groqReqRemaining) groqReqRemaining.textContent = "Aktif (Menunggu data)";
      }

      if (data.remainingTokens !== null && data.limitTokens !== null) {
        if (groqTokenRemaining) {
          groqTokenRemaining.textContent = `${Number(data.remainingTokens).toLocaleString()}
/ ${Number(data.limitTokens).toLocaleString()}`;
        }
        if (groqTokenBar) {
          const pct = Math.max(0, Math.min(100, (data.remainingTokens / data.limitTokens) *
100));
          groqTokenBar.style.width = `${pct}%`;
        }
      } else {
        if (groqTokenRemaining) groqTokenRemaining.textContent = "Aktif (Menunggu data)";
      }
    } else {
      if (groqModelName) groqModelName.textContent = "Belum Dikonfigurasi (.env)";
      if (groqReqRemaining) groqReqRemaining.textContent = "-";
      if (groqTokenRemaining) groqTokenRemaining.textContent = "-";
    }
  } catch (err) {
    console.warn("Gagal memuat kuota Groq:", err);
  }
}

// Handler Tombol Refresh Kuota
if (refreshQuotaBtn) {
  refreshQuotaBtn.onclick = async () => {
    refreshQuotaBtn.disabled = true;
    refreshQuotaBtn.textContent = "Memeriksa...";
    await loadGroqQuota(true);
    refreshQuotaBtn.disabled = false;
    refreshQuotaBtn.textContent = "🔄 Cek Kuota";
  };
}

// Inisialisasi awal
checkBotStatus();
loadKnowledgeList();
loadProductsList();
loadGroqQuota();
setInterval(checkBotStatus, 5000);
setInterval(() => loadGroqQuota(false), 20000);
```

Langkah 15: Panduan Penanganan Kendala Teknis (Troubleshooting FAQ)

Berikut rangkuman solusi atas kendala umum yang sering dihadapi praktikan selama proses instalasi dan pengujian:

Error 'ProtocolError: Target closed' / 'The browser is already running': Terjadi ketika terdapat proses Google Chrome yang stuck dari sesi sebelumnya. Solusi: Klik tombol 'Reset Sesi WA' pada dashboard web lalu hapus folder .wwebjs_auth.

Langkah 16: Menjalankan Project dan Skenario Pengujian

Jalankan server pengembangan dengan mengetikkan perintah berikut di terminal.

```
● ● ●  Terminal
npm run dev
```

Buka http://localhost:3001, klik tombol Mulai Bot, lalu pindai kode QR yang muncul menggunakan fitur Perangkat Tertaut WhatsApp.

A. Hasil yang Diharapkan untuk Praktikum

Praktikum berhasil apabila QR Code muncul, akun WhatsApp berhasil ditautkan, terminal menampilkan Bot WhatsApp siap digunakan, dan pesan dari pengguna mendapatkan balasan Groq AI. Isi bagian ini dengan screenshot QR Code, status bot siap, dan contoh percakapan.

B. Tugas

1. Buatlah website katalog interaktif dengan tema bisnis bebas.
2. Sediakan minimal 4 pilihan kategori produk atau layanan.
3. Tambahkan tombol chat yang langsung mengarah ke WhatsApp pribadi.
4. Buat template pesan otomatis berdasarkan kategori yang dipilih user.
5. Deploy website tersebut ke Vercel hingga bisa diakses publik.
6. Upload tugas project ke repository GitHub.
7. Buka vercel.com dan masuk menggunakan GitHub.
8. Pilih Add New Project.
9. Import repository katalog.
10. Klik Deploy Lalu Buka URL publik hasil deployment.